Athari — Day 1 Report: Setup & Content Model

Owner: Baraka Mitigoa **Project:** Athari (NGO Project & Impact Portal) **Date:** 16 October 2026 **Local site:** https://athari.ddev.site **Admin login (local only):** `admin` / `admin`

---

1. Summary

Day 1 turned an empty folder into a working Drupal 11 site with a complete data model.

- The local toolchain (WSL2, Docker Desktop, DDEV) was installed and verified.
- A Drupal 11.4.10 site was created in `C:\NGO\Athari`, running in DDEV containers, with Drush for the command line.
- Git was initialised, with a proper `.gitignore`, and five logical commits were made.
- Seven contrib modules (plus Devel for development) were installed and enabled.
- Two taxonomies (**Sector**, **Region**) and four content types (**Project**, **Location**, **Partner**, **Report**) were created with all the fields the plan requires.
- Pathauto now produces clean URLs such as `/projects/clean-water-gulu`.
- All configuration is exported to `config/sync`, so the whole site structure lives in Git.

Nothing was left incomplete from the Day 1 checklist. Two things were deliberately deferred: roles/permissions and sample data (Day 2), and a custom theme (Day 3).

---

2. Everything created or changed

2.1 Project scaffolding

Item	What it is	Why it was needed
`.ddev/config.yaml`	DDEV project definition: name `athari`, type `drupal11`, docroot `web`, PHP 8.3, MariaDB 11.8.	Tells DDEV how to build the web and database containers.
`composer.json`, `composer.lock`	Lists every PHP package the site needs (Drupal core, Drush, contrib modules) and pins exact versions.	Anyone can rebuild identical code with `composer install`.
`web/`	The web root. Drupal core lives in `web/core` (not committed; Composer restores it).	Docroot served by DDEV.

Item	What it is	Why it was needed
`web/sites/default/settings.php`	Main Drupal settings file. I added `$settings['config_sync_directory'] = '../config/sync'` and an include of an optional `settings.local.php`.	Moves the config export outside the web root, as the plan requires, and gives a git-ignored place for secrets (Daraja keys on Day 4).
`web/sites/default/settings.ddev.php`	Generated by DDEV: database credentials and hash salt for the local environment.	Lets Drupal connect to the DDEV database. It is git-ignored.
`config/sync/`	Directory of YAML files describing the site's configuration.	Drupal's configuration management target. 138 files at install, 213 now.
`.gitignore`	Excludes `vendor/`, Drupal core and contrib code, uploaded files, `settings.local.php`, scaffold files, `.env`, OS junk and `.claude/`.	Keeps the repo small and prevents secrets being committed.
`scripts/day1-content-model.php`	A Drush script that builds the vocabularies, terms, content types, fields, displays and Pathauto patterns.	Repeatable and reviewable creation of the model. The exported config is the source of truth.
`docs/reports/`	This report and its PDF.	Mandatory daily report.

2.2 Taxonomies

Vocabulary	Terms	Purpose
Sector (`sector`)	Health, Education, WASH, Livelihoods, Protection	The thematic area of a project. Drives the filter and the "by sector" chart on Day 3.
Region (`region`)	4 top-level regions (Central, Eastern, Northern, Western), each with 5 child districts (20 districts)	Geography. Hierarchical, so a project can be tagged to a whole region or to a specific district. Drives the "by region" chart.

Districts used: Central – Kampala, Wakiso, Mukono, Masaka, Mpigi. Eastern – Jinja, Mbale, Soroti, Tororo, Busia. Northern – Gulu, Lira, Arua, Kitgum, Adjumani. Western – Mbarara, Kabarole, Kasese, Kabale, Hoima.

2.3 Content types and fields

All four types have revisions enabled and "Authored by/on" display switched off. "Promoted to front page" and "Sticky" are hidden from the edit forms, because the homepage will be built with Views on Day 3.

Partner (`partner`) — an organisation involved in projects.

Field	Machine name	Type	Notes
Name	`title`	Text (core)	The partner's name.
Logo	`field_logo`	Image	png/jpg/svg/webp, stored in `partners/` .
Website	`field_website`	Link	External URL only.

Location (`location`) — a project site.

Field	Machine name	Type	Notes
Name	`title`	Text (core)	Site name.
District	`field_district`	Entity reference → Region term	Required.
Coordinates	`field_coordinates`	Geofield	Required. Entered as latitude/longitude. Leaflet will plot it on Day 3.

Project (`project`) — the central content type.

Field	Machine name	Type	Notes
Title	`title`	Text (core)	
Description	`body`	Long text with summary	
Budget (USD)	`field_budget`	Decimal (14,2)	Minimum 0.
Start date / End date	`field_start_date` , `field_end_date`	Date	Date only, no time.
Status	`field_status`	List (text)	Planned / Active / Completed. Required.
Sector	`field_sector`	Reference → Sector term	Required.
Region	`field_region`	Reference → Region term	Required. Region or district.
Locations	`field_locations`	Reference → Location nodes	Unlimited. Autocomplete.
Partners	`field_partners`	Reference → Partner nodes	Unlimited. Autocomplete.

I made Locations and Partners multi-value: a real project often has several sites and several partners, and a single value would have been a limitation later.

Report (`report`) — a field report on a project.

Field	Machine name	Type	Notes
Title	`title`	Text (core)	
Project	`field_project`	Reference → Project	Required. Links each report to its project, the basis of all impact totals.
Report date	`field_report_date`	Date	Required.
Beneficiaries reached	`field_beneficiaries`	Integer	Required, minimum 0. Summed on Day 3.
Funds spent (USD)	`field_funds_spent`	Decimal (14,2)	Required. Summed on Day 3.
Narrative	`field_narrative`	Long text	Required.
Attachments	`field_attachments`	File	Unlimited. pdf, doc(x), xls(x), csv, images.

Each field also has a form display (the widget used when editing) and a view display (how it is shown on the page). These are config items too.

2.4 Pathauto URL patterns

Pattern	Applies to	Result
`/projects/[node:title]`	Project	`/projects/clean-water-gulu`
`/locations/[node:title]`	Location	`/locations/gulu-town`
`/partners/[node:title]`	Partner	`/partners/example-org`
`/reports/[node:title]`	Report	`/reports/q1-field-report`
`/sector/[term:name]`	Sector terms	`/sector/health`
`/region/[term:name]`	Region terms	`/region/gulu`

---

3. Modules installed

Module	Version	Purpose	Use in Athari
Admin Toolbar (+ Tools)	3.6.3	Dropdown admin menu.	Faster navigation for staff and for building.
Pathauto	1.15.0	Automatic URL aliases from patterns.	Clean, readable URLs for every content item.
Token	1.17.0	Placeholders such as `[node:title]`.	Required by Pathauto; used in file paths.
Geofield	10.3.4	Latitude/longitude field type.	Stores Location coordinates.
Leaflet	10.4.13	Interactive maps using Leaflet.js.	Project map on Day 3.

Module	Version	Purpose	Use in Athari
Charts (+ Chart.js)	5.2.4	Chart rendering in Views.	Impact dashboard charts on Day 3.
Address	2.0.4	Structured postal address field.	Available for partner/contact details (not yet used on a field).
Devel, Devel Generate	5.5.0	Developer tools and dummy-data generation.	Dev only. Not for production.
Drush	13.8.0	Drupal command-line tool.	Installs, config export, cache clears.

Pulled in automatically: CTools (required by Pathauto), commerceguys/addressing (Address), GeoPHP (Geofield). `datetime_range` and the core Toolbar were also enabled as dependencies.

---

4. Code walkthrough

There is no custom module yet (`athari_donate` starts on Day 4). The only custom code is the build script.

`scripts/day1-content-model.php` runs with `ddev drush php:script`, which boots Drupal and runs the script with full access to its APIs.

- `athari_add_field()` — a helper that does four things for one field: 1. creates **field storage** (`FieldStorageConfig`): the column(s) in the database, type and cardinality; 2. creates a **field instance** (`FieldConfig`): the field attached to one content type, with label, required flag and settings; 3. adds the field to the **form display** (`EntityFormDisplay`) with a widget; 4. adds it to the **view display** (`EntityViewDisplay`) with a formatter. It checks whether each piece exists first, so the script can be re-run safely.
- `athari_ref()` — builds the settings for an entity reference field: the target entity type and which bundles (vocabularies or content types) may be referenced.
- Vocabularies and terms — `Vocabulary::create()` and `Term::create()`. Districts are created with the region as their `parent`, giving the hierarchy.
- Content types — `NodeType::create()` for the four types.
- Pathauto — `PathautoPattern::create()` with a selection condition (`entity_bundle:node`) so each pattern only applies to its own type.

Data flow: the script writes Drupal *config entities* to the database. `drush cex` then exports them to YAML in `config/sync`. On another machine, `drush cim` reads the YAML back, so the same structure appears without re-running the script.

---

5. How to use / test it

1. Start the project if needed: `ddev start` (from `C:\NGO\Athari`).
2. Open https://athari.ddev.site and log in at https://athari.ddev.site/user/login with `admin` / `admin`. (Or run `ddev drush uli` for a one-time login link.)
3. **Taxonomies:** Structure → Taxonomy (`/admin/structure/taxonomy`). Open **Region** to see the regions and their child districts.

4. **Content types:** Structure → Content types (`/admin/structure/types`). Open **Project** → *Manage fields* to see all fields; *Manage form display* to see the edit-form layout.
5. **Try it:** Content → Add content (`/node/add`). - Add a **Partner**, then a **Location** (coordinates e.g. latitude 2.7746, longitude 32.2990 for Gulu). - Add a **Project** that references the location and partner. - Add a **Report** that references the project. - After saving, check the URL: it should look like `/projects/your-title` .
6. **Pathauto:** Configuration → Search and metadata → URL aliases → Patterns (`/admin/config/search/path/patterns`).
7. Bulk-generate aliases for existing content any time: `ddev drush pathauto:aliases-generate create all` .

---

6. Commands run

Command	What it does
`wsl --install --web-download`	Installs WSL2 and Ubuntu (web download after the first attempt failed).
`winget install DDEVFoundation.DDEV`	Installs DDEV on Windows.
`mkcert -install`	Creates a local certificate authority so `https://*.ddev.site` is trusted.
`ddev config --project-type=drupal11 --docroot=web --project-name=athari --php-version=8.3`	Writes `.ddev/config.yaml` .
`ddev start`	Pulls images, builds and starts the web, database and router containers.
`composer create-project drupal/recommended-project:^11 /tmp/athari-build --no-install` (inside the container)	Downloads the Drupal 11 project template into a temporary folder. See Issues.
`composer install`	Installs Drupal core and its dependencies from the lock file.
`ddev composer require drush/drush`	Adds Drush.
`ddev drush site:install standard ...`	Installs Drupal with the Standard profile and creates the admin user.
`ddev drush cex -y`	Exports active configuration to `config/sync` .
`ddev composer require drupal/admin_toolbar drupal/pathauto ...`	Downloads the contrib modules.
`ddev drush en admin_toolbar pathauto token geofield leaflet charts charts_chartjs address devel devel_generate`	Enables them.
`ddev drush php:script scripts/day1-content-model.php`	Builds the content model.
`ddev drush pathauto:aliases-generate create all`	Creates URL aliases for existing content and terms.

Command	What it does
`ddev mutagen sync`	Forces DDEV's file sync between container and Windows.
`git init -b main`, `git add`, `git commit`	Version control.

---

7. Drupal concepts explained

- **Docroot and Composer:** Drupal's public files live in `web/`; everything else (`vendor/`, `config/`, `composer.json`) stays outside the web server's reach, which is more secure. Composer downloads and versions all code; you never download modules by hand.
- **Content type, node, field:** A *node* is one piece of content. A *content type* (bundle) defines what fields a node has. A *field* has two halves: *storage* (database definition, shared across types) and an *instance* (its use on one bundle).
- **Entity reference:** A field that points to another entity (a taxonomy term or node) by ID. Project → Sector is a term reference; Report → Project is a node reference. This is how Drupal models relationships, and what Views aggregates over on Day 3.
- **Taxonomy:** Vocabularies of terms for classification. Region is hierarchical (parent/child) to model region → district.
- **Form display vs view display:** The form display decides which widget edits a field; the view display decides how the field is shown to visitors. Both are configuration.
- **Configuration management:** Site structure (types, fields, views) is stored as config in the database and can be exported to YAML (`cex`) and imported (`cim`). *Content* (nodes, terms) is not config, so it is not in Git. Day 2 seeds content with a script instead.
- **Pathauto and tokens:** Pathauto builds URL aliases from patterns; tokens like `[node:title]` are replaced with real values.
- **Geofield:** Stores geometry (here a point). The Leaflet module reads it to draw maps.
- **DDEV and Mutagen:** DDEV runs Drupal in Docker containers. On Windows it uses Mutagen to sync files between Windows and the container for speed; the sync is asynchronous, so a few seconds may pass before changes appear on the other side.

---

8. Issues and fixes

Issue	Cause	Fix
Docker, WSL2 and DDEV were missing at the start.	Fresh machine.	Installed them following the steps given (WSL2 → Docker Desktop → DDEV).
`wsl --install` failed: "connection with the server was reset".	Network drop during download.	Retried with `--web-download`; completed.
`winget install DDEV.DDEV` said "No package found".	DDEV's winget ID changed.	`winget search ddev` showed `DDEVFoundation.DDEV`.

Issue	Cause	Fix
`mkcert -install` printed a Java `keytool` error for Android Studio.	mkcert tried to add its certificate to Android Studio's Java store, which needs admin rights.	Ignored: the Windows trust store (used by browsers) was updated successfully.
`ddev composer create` wants an empty folder, but the folder already held `CLAUDE.md` and DDEV files.	Composer's create-project refuses non-empty targets.	Created the project in a temp folder inside the container, copied `composer.json` and `composer.lock`, then ran `composer install`.
First `drush/drush` install did not run.	A PowerShell pipe to `tail` (a Linux command) errored.	Re-ran without the pipe.
Some `ddev drush` calls hung when run from PowerShell.	Drush waited on input from the console.	Used `ddev exec "drush … < /dev/null"` and killed the stuck process.
`git status` showed no config changes straight after `drush cex`.	Mutagen had not synced the new files to Windows yet.	Ran `ddev mutagen sync` before committing.
URL aliases were missing for taxonomy terms.	The terms were created before the patterns existed.	Ran `pathauto:aliases-generate create all`.
Git warns "LF will be replaced by CRLF".	Windows line-ending setting.	Harmless; not changed, as git config must stay as is.

Note: git's `user.name` is `bmitigoa` (not "Baraka Mitigoa"). Commits carry that name. To change it later: `git config user.name "Baraka Mitigoa"`.

---

9. Git log

```
6ec7121 chore: scaffold Drupal 11 project with DDEV and Drush
7e60ee5 config: initial Drupal standard profile configuration export
22db358 feat: add contrib modules (admin toolbar, pathauto, token, geofield, leaflet, charts,
address, devel)
cdd9936 chore: add Day 1 content model build script
814471a config: add sector/region taxonomies, project/location/partner/report content types and
pathauto patterns
```

A final `docs: day 1 report` commit adds this report and the PDF.

---

10. Next day (Day 2 — Roles, Workflow & Sample Data)

- Create roles: Field Officer, M&E Reviewer, Admin, plus one test user per role.
- Enable Content Moderation and Workflows with Draft → In Review → Published → Archived on Reports.

- Set permissions: field officers create/edit their own reports and submit for review, reviewers approve or reject, admins manage everything.
- Seed realistic data: about 10 projects, 15 Ugandan locations with real coordinates, 5 partners and 30 reports.